

Trabajo Práctico N°1

Base de Datos 2 - FoodStore

Estudiante: Enzo Giaquinta

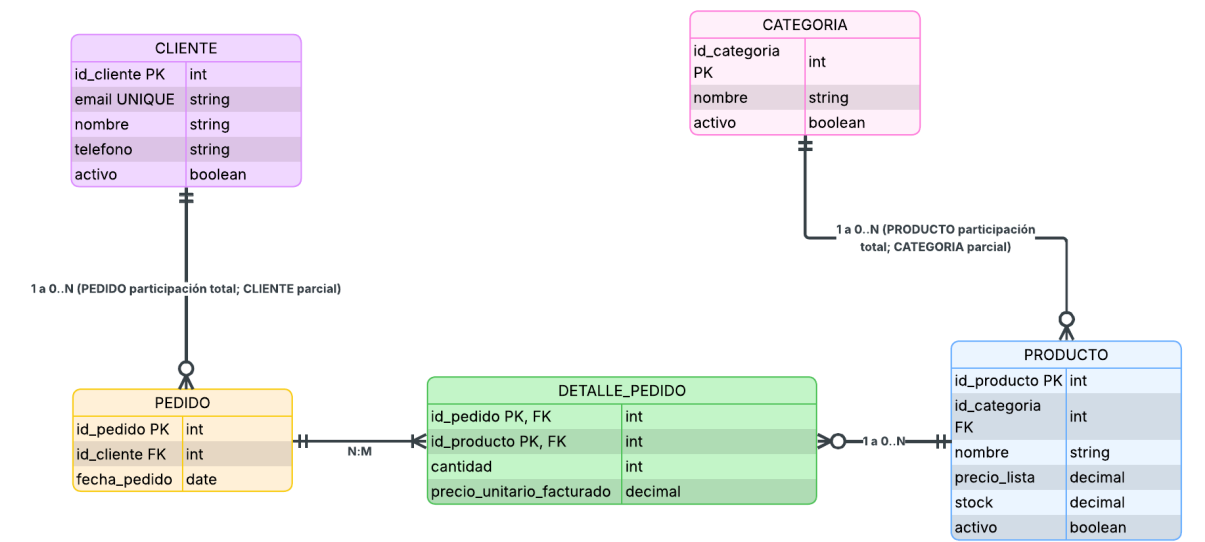
Legajo: 53884

Comisión: 4

Consignas

Parte 1

Diagrama ER (FoodStore):



Diccionario de entidades y atributos:

Entidad	Atributo	Tipo de dato	Clave / restricción	Descripción
CLIENTE	id_cliente	int	PK	Identificador único del cliente.
	email	string	UNIQUE, NOT NULL	Correo electrónico único que identifica al cliente.
	nombre	string	NOT NULL	Nombre del cliente.
	telefono	string	—	Número de teléfono del cliente.

	activo	boolean	NOT NULL	Indica si el cliente se encuentra activo.
CATEGORIA	id_categoria	int	PK	Identificador único de la categoría.
	nombre	string	NOT NULL	Nombre de la categoría, por ejemplo, pizzas o bebidas.
	activo	boolean	NOT NULL	Indica si la categoría está activa. Permite conservar categorías dadas de baja.
PRODUCTO	id_producto	int	PK	Identificador único del producto.
	id_categoria	int	FK, NOT NULL	Identificador de la categoría a la que pertenece el producto.
	nombre	string	NOT NULL	Nombre del producto.
	precio_lista	decimal	NOT NULL, ≥ 0	Precio actual de lista del producto.
	stock	decimal	NOT NULL, ≥ 0	Cantidad disponible del producto.
	activo	boolean	NOT NULL	Indica si el producto está activo. Permite conservar productos dados de baja.

PEDIDO	id_pedido	int	PK	Identificador único del pedido.
	id_cliente	int	FK, NOT NULL	Identificador del cliente que realizó el pedido.
	fecha_pedido	date	NOT NULL	Fecha en la que se realizó el pedido.
DETALLE_PEDIDO	id_pedido	int	PK, FK	Identificador del pedido al que pertenece la línea.
	id_producto	int	PK, FK	Identificador del producto incluido en la línea del pedido.
	cantidad	decimal	NOT NULL	Cantidad de unidades del producto solicitadas en el pedido.
	precio_unitario_facturado	decimal	NOT NULL	Precio por unidad que se facturó en ese pedido. Conserva el precio histórico aunque cambie precio_lista.

Preguntas guía:

- ¿Por qué la relación entre producto y pedido no puede resolverse como una relación binaria simple 1:N? ¿Qué información se perdería si lo intentaras?

Porque un pedido puede tener varios productos y un producto puede aparecer en muchos pedidos, por lo que la relación es N:M. Por esto se usa la tabla intermedia "Detalle_Pedido". Esta tabla intermedia guarda la cantidad de productos y el precio

unitario del pedido, por lo que se perderían estos datos al convertirlo en una relación 1:N.

- ¿Qué entidad tiene participación parcial en la relación con categoría, y cuál tiene participación total? ¿Qué pasaría si invirtieras esa lectura?

“Producto” tiene participación total, porque todo producto debe pertenecer a una categoría. “Categoría” tiene participación parcial, porque puede existir una categoría sin productos. Si se invirtiera la lectura, se estaría permitiendo que un producto no tenga categoría y obligando a que toda categoría tenga al menos un producto, contradiciendo la primera regla de negocio.

- ¿Alguno de tus atributos podría descomponerse en partes más simples (por ejemplo, un nombre completo en nombre y apellido)? Justificá tu decisión de dejarlo compuesto o de separarlo.

Se pueden descomponer algunos atributos en partes más simples, como el nombre cliente, pero en las reglas de negocio no se especifica lo contrario, por lo que decidí dejarlas como están. Esto también disminuye la complejidad de las tablas, al disminuir la cantidad de campos distintos en una entidad.

Parte 2

Modelo relacional:

- **cliente** (id_cliente PK, email UNIQUE, nombre, telefono, activo).
- **categoría** (id_categoria PK, nombre, activo).
- **producto** (id_producto PK, id_categoria FK → categoría, nombre, precio_lista, stock, activo).
- **pedido** (id_pedido PK, id_cliente FK → cliente, fecha_pedido).
- **detalle_pedido** (id_pedido PK/FK → pedido, id_producto PK/FK → producto, cantidad, precio_unitario_facturado).

Preguntas guía:

- ¿Qué pasaría con la integridad de tus datos si la tabla intermedia no incluyera ambas claves foráneas como NOT NULL?

Si las claves foráneas no fueran NOT NULL, podrían existir registros en “Detalle_Pedido” sin un pedido o sin un producto asociado. Esto generaría registros incompletos y afectaría la integridad de la relación entre ambas tablas.

- Si el pedido pudiera existir sin ningún producto asociado (por ejemplo, un pedido recién iniciado), ¿cambia esto la participación que definiste en la Parte 1 para la relación con la tabla intermedia? Explica el impacto.

Sí cambia la participación. "Pedido" tendría participación parcial respecto de "Detalle_Pedido", porque podría existir un pedido sin ningún detalle de producto asociado, y por ende, ningún producto. Sería como un pedido vacío.

Parte 3

1. Para determinar la clave candidata, sabemos que `n_pedido` no es suficiente, ya que un pedido puede tener varios productos, e incluso en el caso de la tabla se repite el pedido 1. Pero si combinamos el `n_pedido` con el producto, obtenemos una clave candidata. Las mismas reglas de negocio se menciona que en un pedido no se repite el mismo producto.
2. Dependencias Funcionales:
 - a. `Nº pedido` -> `Fecha`, `Cliente`, `Forma de pago`. Cada pedido tiene una única fecha, un único cliente y una única forma de pago.
 - b. `Producto` -> `Categoría`. Cada producto pertenece a una única categoría.

Con la nueva clave candidata (`n_pedido` , `producto`):

- c. clave candidata -> `precio_unitario`: el precio cambia con el tiempo, por lo que depende del pedido concreto.
 - d. clave candidata -> `cantidad`: cantidad de un producto vendido, de un pedido específico.
 - e. (`precio_unitario`, `cantidad`) -> `subtotal`: precio final luego de calcular `precio_unitario` x `cantidad`.
3. La tabla está en 1FN, ya que cada celda cumple con la atomicidad, o sea, que cada cuadrado o celda contiene un único valor y no una lista de valores.
4. No cumple 2FN, ya que atributos como `categoría` o `precio_unitario` dependen de `producto`, mientras que `fecha` o `cliente` dependen solamente de `n_pedido`. Por eso descompongo en tablas:
 - a. Pedidos (`n_pedido`, `fecha`, `cliente`, `metodo_pago`).
 - b. Productos (`producto`, `categoría`).
 - c. Detalle_Pedido (`n_pedido`, `producto`, `precio_unitario`, `cantidad`, `subtotal`).
5. Si bien, luego del 2FN las tablas mantienen ninguna dependencia transitiva, la única que quizás podría ser es el `subtotal`, que depende de 2 atributos no claves (`cantidad`, `precio_unitario`). Yo lo mantendría, pero para reglas del trabajo debería eliminarse, de esa manera se mantiene la normalización 3FN.
6. En las tres tablas se cumple BCNF. En Pedidos el determinante `n_pedido` es clave primaria de la tabla. En Productos el determinante `producto` es clave primaria de la tabla. En detalle pedido el determinante es la clave compuesta {`n_pedido`, `producto`}.
7. Conjunto Final de Tablas Normalizadas (las claves primarias están indicadas en negrita):
 - a. PEDIDOS (**`n_pedido`**, `fecha`, `cliente`, `forma_pago`).

- b. PRODUCTOS (**producto**, categoría)
- c. DETALLE_PEDIDO ({**n_pedido**, **producto**}, precio_unitario, cantidad)

Preguntas de integración:

- Tengo 3 tablas en el ER de la parte dos que son las mismas de esta tabla luego de normalizarla. No son exactamente iguales, incluso siendo que aquí termine poniendo categoría dentro de productos, siendo que en la parte 2 la hice una tabla a parte. También puse muchos más atributos en el ER que los que tengo en las últimas tablas. Creo que la diferencia fundamental está en la cantidad de datos. En esta parte 3 vimos un caso real de un historial de pedidos, con los datos que esta tienda utiliza y registra. Pero la forma en que registran sus pedidos no necesariamente es la misma que la de otro restaurante. Esto hace que si luego de normalizar esta tabla, vayamos a otra de una tienda distinta, quizás tengamos más o menos trabajo. El hacerlo desde el principio nos da la libertad de imaginar y elegir qué datos pueden ser necesarios y usados de determinada manera.
- Creo que la necesidad de almacenar el subtotal depende de lo que quiera la empresas o restaurante, pero para registros históricos, seguramente se facilite automatizar un recuento de ganancia si ya se tiene almacenado el subtotal, pensando en registros con miles o decenas de miles de pedidos, que si se tuviera que calcular para cada uno, tardaría mucho más. Si se mantiene se viola la 3FN, pero a criterio mío lo dejaría por la ventaja en rendimiento. Aunque se podría pensar que un atributo menos es menos espacio ocupado, la ventaja en rendimiento se nota en otros procesos, como el que mencioné de recuento de ganancias.